

AI-Native 组织考核框架

AI-Native 组织考核框架

一句话：考核的对象是人，但量的是贴在每件工作上的标签——IC / Builder / Coach / DRI——各自该量的东西。四者服务同一个目标：**淘汰协作冗余，不是惩罚人**。绝对标准、不末位淘汰、不按比例。

一、设计原则

- 1. **淘汰协作冗余，不是惩罚人。** 冗余 = 同一份判断被重复生产，不是”存在一个协作节点”。
- 2. **绝对标准、不末位淘汰、不按比例。** 按比例强淘汰会把协作推成竞争。
- 3. **标签贴在工作单元上，不是职位。** 同一个人一周里干的是混合，按每件工作该用的尺各自量。
- 4. **AI 给 what + how，人给 why。** 越往上（IC → Coach → DRI），可量化成分越少、人的判断与担当越多。
- 5. **评分只做诊断，不做裁决。** “谁留谁走”是具名的人做的判断，机器不自动开除。

二、对象模型：人 / 工作单元 / 标签

考核主对象是人。工作单元 = 一件端到端的事。标签由工作流自动贴在工作单元上、不是自己选——你不能挑一个好藏的标签来躲考核。

标签	砍的冗余	考核的货币	资格线（同形）
IC	干活不用一堆人	自己的闭环 + 自己变自主	一周不抛弃 = 没在认真探索
Builder（IC 子类）	让交付不依赖某个具体的人	每周固定吞吐 + 可插拔性	变得不可插拔（context 进脑子）= 设计失败
Coach	未来不用重复求助	写下的、能被复用的判断	一周没判断进共享知识库且无人复用 = 没

标签	砍的冗余	考核的货币	资格线（同形）
			做教的那摊
DRI	决策不用委员会 / 审批链	决策质量 + 校准 + 担当	一周期没接一个难决策 = 没在真做 DRI

三、三个标签各自的考核

3.1 IC —— 自己的闭环 + 变自主

三个维度：探索进展 / 闭环效率 / 判断力。两个核心信号：

- 一周不抛弃 = 没在认真探索（果断抛弃假设不是没产出，是把范围越缩越准）。
- 向单元外求助逐周减少 = 强正向加分——量的是”你向外求助在变少”（你在变自主）。辅助别人永远是加分项，绝不是减分。

「AI 介入度」L1/L2/L3 轴（正交于标签）——同一件工作，除了贴标签，再标一档 AI 介入度：

档	含义	人在干嘛
L1 AI 辅助	Copilot 式补全 / 小函数	人主导，AI 打下手
L2 人机协同	人拆解任务、agent 干一块	人编排 + 验证
L3 Agentic	agent 自主跑完	人只澄清需求 + 验收

考核看的不是”用没用 AI”，是这个人/团队的工作在 L2+L3 的占比是否逐周上升（真在把活交给 agent、自己上移到编排和判断），而不是长期卡在 L1（只把 AI 当打字员）。工作法：IC 的动作从”亲手写”变成 **delegate** → **review** → **own**（派给 agent → 审 → 担责），**review** 成为新瓶颈。

当 IC 从”亲手写代码”转成”驾驭一队 agent”，考核对象从个人代码变成人 + agent 队列产出：部署频率必须配返工率、agent 任务成功率 / 单任务成本 / 升级频率、真正不用人接手的比例、人审队列深度。



AI 介入度 × agent 自主度 —— 两把尺对齐

3.2 Coach —— 按可测性切两半

Coach 不是传统经理。时间一半给 agent、一半给人，而这两半的**可测性正好相反**，所以用相反的工具量。

- **教 agent（可硬测）**：agent 拿不准时把 edge case 推上来，coach 写一个判断，**这条进共享知识库、所有 agent 都读**。这半全程有日志，可量化：判定线是「你写一次、下游读 N 次自己跑通、不必再回来问你」（杠杆）对「下游每次撞同类还得重新找人问一遍」（冗余）。用本窗口判断的复用率 + 该类问题的人类升级率下降来衡量。
- **接住人（不可测）**：一个人快撑不住时反而看着最稳、不会主动求助。靠平时一起干活攒下来的感知去接住他。这半**一旦去量就摧毁它赖以成立的坦诚**——所以个人层面一个数都不打，只通过结果、单元是否健康（没在装、没 burnout）、以及”你真正跟谁学的”这类低频反馈慢慢浮现。

在本组织，coach 的”接住人”那摊**主要面向 Builder fleet**（见第八节）：教的那半（建可复用 playbook）放大，接的那半收缩。

3.3 DRI —— 决策质量 / 校准 / 担当

DRI 是结果归属角色，不是过程角色，所以换一根脊柱。

最大的坑：用结果好坏考核 DRI。 大合同没签下、产品方向错了，运气和外部因素占一大半。按赢/输打分会因坏运气罚掉做对决策的人，还逼 DRI 只接稳的活。**判断按”下注质量”算，不按”骰子点数”算。**

四条脊柱（全部可审计）：

1. **决策质量（事前）** ——在当时可知信息下，这个决定对不对？结果出来前就能审。

2. **校准**——拍板时给出概率分布；跨多次决策看校准（你说 70% 会发生的事是不是真的约 70% 发生）。这是唯一能在异质、稀疏事件上聚合的量，单独解决”高风险事件太少没法量化”。
3. **担当**——出错时：早暴露、如实上报、抽对资源、把教训写回知识库，还是藏、僵、甩锅。这对应”人给 why”。
4. **真用了权（资格线）**——一周期没接一个艰难的、本可甩锅但接住了的决策 = 没在真做 DRI。

接进哲学：DRI 砍的是**决策协调冗余**——一个具名全权负责人 = 干掉委员会、审批链、会签。所以一个要靠一堆会和会签才能拍板的 DRI，是在重新制造官僚冗余。

授权用 **L1—L5 五级自主度阶梯**（按任务选合适级，不是越高越好）：L1 单文件补全 / L2 对话式人全程把舵 / L3 交一个任务收一个 PR、人异步审 / L4 agent 自己从待办挑活、人审产出 / L5 多 agent 编排、人在编排层监督。关键：**没有哪一级是”无人审自动合”**，人类监督只是从**逐行 → 逐 PR → 逐产出 → 逐编排层一路上移，闸门一直在人手里**。review 负担从 L3 起就可能成瓶颈——DRI 的职责之一就是按”审得过来”动态校准每类工作的自主级。

为什么 DRI 不能交给 agent：行业里把客服大规模交给 AI 的公司，在高判断 / 情绪类工作上崩盘后又回招了人（AI 顶一线、人接升级 / 欺诈 / 销户）。这正是”高风险结果具名负责”必须留给人的实证。

四、从标签到人

人是主对象，标签是这个人 workflows 上的切面。人层面的考核是**组合，不是一个合成分**（不同标签的指标单位不一样，加不起来）：

- 每个标签各有一条绝对地板。**过关** = 在你真正做过的标签上各自踏过地板，加上慢层（结果、单元健康）没暴雷。
- **淘汰触发** = 在你实际工作的主导标签上、滚动窗口内持续踏不过地板——不是排名、不是合成分垫底。节点：两天黄灯（中性的”你卡在哪”） / 一周橙灯（换方向或角色） / 两周红灯（退出）。
- **流动治理**：同一人在多个标签间流动时各自的账并行累加，切标签洗不掉低产。
- **新人两天缓冲**：入职两天不触发任何淘汰节点，第二天结束由新人自己决定接不接受这套制度。

五、考核哲学的统一

三个标签服务同一个”淘汰协作冗余”，打在三条不同战线：

- **IC = 吞吐**：把活闭环掉、不用一堆人。
- **Coach = 杠杆**：把判断生产一次、沉淀下来、众多 agent 复用——消除未来的冗余。

- **DRI = 担当**：用一个具名全权负责人替掉整条审批链——消除决策协调的冗余。

冗余的重定义：协作冗余 = 同一份判断被重复生产，不是”存在一个协作节点”。一次性供给可复用判断（coach）、一个具名负责人替掉委员会（DRI），都是在消除冗余，不是冗余本身。

六、考核仪表盘

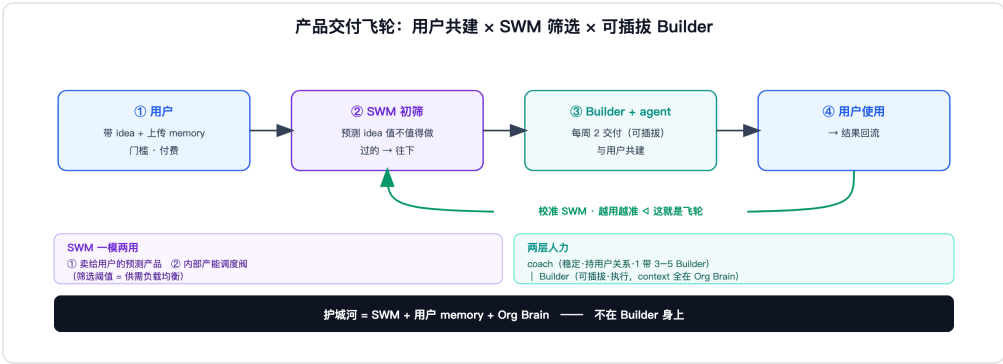
它不是外挂在公司上的 HR 工具，而是**把组织的社会预测内核掉头对准公司自己的人类操作视图**：读同一份事件日志，算每个人/单元的过载风险、流失风险、速度漂移，渲染成每个人的工作组合时间线 + 地板灯带（绿=过 / 黄=滚动窗口偏低，触发中性对话，不是判决）+ 不归因到个人的单元健康图。

最关键的产品决策：它是**诊断器，不是裁决机**。出信号、出黄灯、出冗余地图，但”该不该让某人走”是一个具名的人的判断。一旦让它自动开除，就废了”人给 why”。

七、招聘侧

把”申请加人前先证明 AI 干不了”写成编制默认规则——这是”淘汰协作冗余”在招聘侧的预防版：冗余还没产生就不引入，比事后淘汰体面。

八、本组织的人力模型：Builder × coach + 产品交付飞轮



产品交付飞轮

飞轮：用户带着想法、上传自己的 memory（既是准入门槛也是付费）→ 预测引擎做初筛（预测的是**用户反应 / 早期信号**，不是”这个想法最终能不能成”）→ 过的，由 Builder + agent 每周固定交付若干、与用户共建 → 用户使用、反应回流 → 预测引擎越校越准 → 回到顶。预测引擎一
模两用：既是卖给用户的产品，又是内部产能的调度阀。

为什么这里能”大量 + 随时插拔”用 Builder：判断外置给预测引擎、记忆外置给共享知识库和用户上传的 memory、执行用 agent 放大——Builder 因此不持有护城河，可以商品化、可以插拔。但要分清：真正抗复制的护城河在 coach 层 + 沉淀下来的可复用判断，不是 memory 或预测引擎本身（用户的 memory 法律上归用户、可携带可撤回；初筛能力本身竞品几周可复制）。可插拔的是执行，不是关系。

Builder 标签：本质是一个 IC 子类的执行单元（Builder 本人 + 一队 agent），跑已筛选、已切好的交付。考核看闭环吞吐 + AI 介入度。判断与探索维压**低但不归零**——留一条窄判断轨（在已筛的件内做局部判断、把可复用判断写进知识库），否则喂不了护城河、还会加速”培养断裂”。“你不在也能转”从加分项升成资格硬门：变得不可插拔 = 设计失败。

coach 重定向：在本组织，coach 的”人那摊”主要面向 Builder fleet。教的那半（建并维护 Builder playbook，让任何 Builder 几天内插上来就能产出）放大——这正是”插拔”成立的成因；接的那半收缩（对可插拔的人，组织选择”换”而非”深接”）。容量上限约 1 个 coach 带 3—5 个 Builder。这就是”招 Builder 的上限 = coach 容量”的原因——coach 是这层里唯一不可插拔的人。

解”插拔 vs 共建”的张力：用户共建的对象是稳定的 coach + 预测引擎层，不是某个具体 Builder。coach 持有用户关系的线，Builder 是线后面可换的手。于是人力分两层：稳定层（coach + 引擎，持关系、带 fleet、沉淀判断）/ 弹性层（Builder + agent，跑已筛的交付）。

落地前提与排序（不可跳）：

- **第 0 步是阻断式闸门：**先拿一批自有的、有真实反馈数据的历史案例做盲测，证明”模拟反应与真实反应逐条对得上的比例显著高于通用基线”。这一步没过之前，Builder 这一层一行都不许 scale，也不要求用户上传隐私级 memory。给一台没点着的发动机装涡轮就是反了。
- 通过后再依次：把”卖给用户的预测”和”Builder 排产”用不同触发器解耦（排产在校准证明前用确定性规则兜底，别让”用户的想法能不能做”取决于这周 Builder 忙不忙）；落实 memory 的所有权 / 删除 / 可携带的法律地基与用工合规；冷启动先用”无需上传 memory、只跑公开 demo”换第一批真实回流；把核心叙事从”护城河在引擎+memory”改写为”护城河在 coach 层 + 可复用判断”；实测真实周吞吐再回填产能模型；三绿灯（memory 确有因果提升 + 实测吞吐达标 + 稳态毛利达标）才允许扩 fleet 超过 1:5。
- **盲测通过后二选一，不能既要又要：卖结果（砍掉”共建”承诺，价值锚到准、快、便宜）或卖关系（承认关系不可插拔，coach 成为产品的硬产能天花板，毛利受限）。**
- **停止信号（出现即重做，非微调）：**盲测在没见过的新场景上与通用模型无显著差异；只在”被扶过”的样本上才准（飞轮”越用越准”是自我实现假象）；前若干目标用户里愿在未验证时上传隐私 memory 的不足两成；外部团队几周内即可复制初筛；某次真实运行中用户在关键时刻没被接住、当场识破”共建”是话术；有人提议”放松可插拔、让 Builder 把 context 攒进脑子”（前提已坍塌）；稳态毛利低于门槛（这是代运营不是软件）。

九、外部实践印证

- **Netflix「Keeper Test」**：每个经理问“这人要走我会不会拼命留”，不会就给优厚遣散；绝对标准、不搞末位、不搞 PIP。逻辑是“还行的人也在拉低团队密度、给周围施加成本”——几乎逐字命中“淘汰协作冗余”。
 - **某万人研发团队的 L1/L2/L3 AI 介入度分级 + 成熟度模型**：用“L2+L3 级需求占比”当牵引指标推动转型；用严口径统计 AI 代码生成率。给了“人-agent 分工可量化分级”的现成做法。
 - **微软的 senior 带 junior 计划**：明确比例约 3:1 到 5:1，并把 senior 角色定义为“教判断的人”——给了“招 Builder 的上限 = coach 容量”一个现实锚。
 - **一家约五十人、估值数亿美元的 AI 编码公司**：产品工程师只 review 高风险改动、把判断当治理沉淀下来，是 coach + 仪表盘双角色的现实样本。
 - **AI agent 自主度的五级阶梯（业界框架）**：明确“越高不一定越好，按任务选合适级”，且各级都保留人审批——本框架第 3.3 节直接采用。
 - **多家公司客服大规模 AI 化后回招人**：印证高判断 / 情绪类工作必须留给人，即第 3.3 节“DRI 不交给 agent”。
-

核心判断：这套框架的引擎是把判断、记忆、执行三件套搬出人脑，从而让执行层可商品化、让稳定的 coach + 预测层持有真正的护城河。能不能落地，全押在一个前提上——**预测引擎的校准能不能被证明**。所以路线是“先证明、再 scale”，不是“先铺人、边走边看”。